

EECE 7205: Fundamentals of Engineering Algorithms

Binary Trees In-Class Work

Natan Blumberg

Fall 2026

Array used in every problem: $\text{arr} = \{5, 12, 8, 21, 3, 17, 9\}$ ($n = 7$)

Problem 1: Creating the Tree

(a) Add the elements of arr one at a time. For each element write its index, its parent index $(i - 1)/2$, whether it is a left or right child, and the rule used.

Drawings: Appendix, Figure 1

Step	Element	Index i	Parent index	Parent value	Left / Right	Rule(s)
1	5	0	None (root)	None (root)	None	C1
2	12	1	0	5	Left	C2, C3
3	8	2	0	5	Right	C2, C4
4	21	3	1	12	Left	C2, C3
5	3	4	1	12	Right	C2, C4
6	17	5	2	8	Left	C2, C3
7	9	6	2	8	Right	C2, C4

Final tree in level order: 5, 12, 8, 21, 3, 17, 9

(b) Which nodes have NULL children? Which rule tells you this, and why?

The nodes 21, 3, 17, and 9 all have no children. Rule C5 tells us this. C5 states, “If a child index $2i + 1$ or $2i + 2$ is n or more, that child is NULL.” These nodes sit on indices 3 to 6. When doing the math to locate their children, $2i + 1$ and $2i + 2$, we get an index of $n = 7$ or more. Thus, they have NULL children.

(c) For your final tree (P7), give each of the following.

Root	5
Leaves	21, 3, 17, 9
Internal nodes	5, 12, 8
Number of edges	6
Height	2
Depth of 17	2
Ancestors of 3	12, 5
Descendants of 12	21, 3
Subtree rooted at 8	8, 17, 9
Path from 5 to 9	5, 8, 9
Length of that path	2

(d) Check the properties, then say whether the tree is full, complete, or both, using rule C6.

Property	Formula	Work	Result
Max nodes on level 2	2^L	$2^2 = 4$	4 (Full)
Max nodes for your height	$2^{H+1} - 1$	$2^{2+1} - 1 = 7$	7 (At max)
Leaves = (nodes with two children) + 1	count + 1	$3 + 1 = 4$	4
Minimum height for $N = 7$	$\lfloor \log_2 N \rfloor$	$\lfloor \log_2 7 \rfloor = \lfloor 2.807 \rfloor = 2$	2

Full, complete, or both? Explain using C6:

The tree is both full and complete. By C6, rules C2 to C4 place each element at the next index, so the tree fills level by level, left to right, with no gaps. That makes it complete. It is also full: every internal node (5, 12, 8) has exactly two children, and every level holds its maximum number of nodes, since $n = 7 = 2^{H+1} - 1$.

Problem 2: Traversing the Tree

(a) **Preorder traversal (Node, Left, Right).** For every node, say why it is visited at that moment and name the rules used (T1, T2, T6).

Visit #	Node	Why it is visited at this moment	Rule(s)
1	5	Get to the root and visit it first, before its left subtree (12, 21, 3).	T2
2	12	Arrive at 12 and visit it, before going to its left subtree (21).	T2
3	21	Arrive and visit. Then see that both children are NULL, return to 12.	T1, T2, T6
4	3	Back at 12, go right. Visit, then see that both children are NULL, return.	T1, T2, T6
5	8	Left side of 5 is completed, go right (8). Visit 8.	T2
6	17	Arrive and visit, see both children are NULL, return to 8.	T1, T2, T6
7	9	Back at 8, go right. Visit, then see both children are NULL, return.	T1, T2, T6

Preorder result	5, 12, 21, 3, 8, 17, 9
------------------------	------------------------

(b) Inorder traversal (Left, Node, Right), naming the rules (T1, T3, T6).

Visit #	Node	Why it is visited at this moment	Rule(s)
1	21	Go left from 5, and again left of 12, hit 21, see no left child, visit it.	T1, T3, T6
2	12	Its left subtree is completed, visit 12, go right.	T3
3	3	Left child is NULL, visit, go right, right child is NULL.	T1, T3, T6
4	5	The left subtree is completed, so visit the root and go right.	T3
5	17	Go to 8, it has a left child, go to that child, it has no left child, visit.	T1, T3, T6
6	8	Left subtree is completed, visit 8, go right.	T3
7	9	Left child is NULL, visit, right child is also NULL.	T1, T3, T6

Inorder result	21, 12, 3, 5, 17, 8, 9
-----------------------	------------------------

(c) Postorder traversal (Left, Right, Node), naming the rules (T1, T4, T6).

Visit #	Node	Why it is visited at this moment	Rule(s)
1	21	Both children are NULL, so nothing to wait for, visit.	T1, T4, T6
2	3	Both children are NULL, visit.	T1, T4, T6
3	12	Both left and right children are finished, visit.	T4
4	17	Both children are NULL, visit.	T1, T4, T6
5	9	Both children are NULL, visit.	T1, T4, T6
6	8	Both left and right children are finished, visit.	T4
7	5	Both subtrees are finished, visit.	T4

Postorder result	21, 3, 12, 17, 9, 8, 5
-------------------------	------------------------

(d) Level order traversal using a queue (T5). Complete the queue table.

#	Queue before	Remove and visit	Add children (left, right)	Queue after	Output so far
1	[5]	5	12, 8	[12, 8]	5
2	[12, 8]	12	21, 3	[8, 21, 3]	5, 12
3	[8, 21, 3]	8	17, 9	[21, 3, 17, 9]	5, 12, 8
4	[21, 3, 17, 9]	21	None	[3, 17, 9]	5, 12, 8, 21
5	[3, 17, 9]	3	None	[17, 9]	5, 12, 8, 21, 3
6	[17, 9]	17	None	[9]	5, 12, 8, 21, 3, 17
7	[9]	9	None	[] (empty)	5, 12, 8, 21, 3, 17, 9

Level order result	5, 12, 8, 21, 3, 17, 9
---------------------------	------------------------

(e) Why is the level order result identical to the original array? Which traversal visits the root last, and why?

Why level order = original array:

Level order matches the original array because rules C2 to C4 built the tree by placing the array elements level by level, left to right, by index. T5 reads the tree back in that same order, level by level and left to right, so it returns the elements in their original index order.

Traversal that visits the root last, and why:

Postorder visits the root last. Its order is Left, Right, Node, so a node is visited only after both of its subtrees are finished. The root's subtrees contain every other node, so the root has to come after all of them.

(f) Search for 3 and for 4 using level order. How many comparisons does each search need?

Search for	Nodes compared (in order)	Found?	Comparisons
3	5, 12, 8, 21, 3	Yes	5
4	5, 12, 8, 21, 3, 17, 9	No	7

Problem 3: Adding Elements to the Tree

(a) **Case I1:** the tree is empty and you insert 5. Where does 5 go, and which rule applies?

It goes to the top and becomes the root. This is rule I1.

Summary table. Fill in as you complete parts (b) to (e).

Drawings: Appendix, Figure 2

Insert	Nodes checked (in order)	Rule at each node	Parent	Left / Right	Level order after
11	5, 12, 8, 21	(I2), I5, I5, I5, I3	21	Left	5, 12, 8, 21, 3, 17, 9, 11
2	5, 12, 8, 21	(I2), I5, I5, I5, I4	21	Right	5, 12, 8, 21, 3, 17, 9, 11, 2
14	5, 12, 8, 21, 3	(I2), I5, I5, I5, I5, I3	3	Left	5, 12, 8, 21, 3, 17, 9, 11, 2, 14
6	5, 12, 8, 21, 3	(I2), I5, I5, I5, I5, I4	3	Right	5, 12, 8, 21, 3, 17, 9, 11, 2, 14, 6

(b) **Insert 11 into the tree from Problem 1. List every node you check in level order, whether its left and right children are present, and the rule applied (I2 to I5). Where does 11 go?**

#	Node checked	Left child	Right child	Decision	Rule
1	5	12	8	Both children present, add to queue, move on.	I2, I5
2	12	21	3	Both children present, move on.	I5
3	8	17	9	Both children present, move on.	I5
4	21	NULL	NULL	Left is empty, so insert 11 there.	I3

11 goes to	The left child of 21
-------------------	----------------------

(c) **Now insert 2. Why does 2 go to a different side of the same parent? Which rule decides this?**

#	Node checked	Left child	Right child	Decision	Rule
1	5	12	8	Both children present, add to queue, move on.	I2, I5
2	12	21	3	Both children present, move on.	I5
3	8	17	9	Both children present, move on.	I5
4	21	11	NULL	Left child is present, right is NULL, so insert 2 there.	I4

Why a different side, and which rule:

2 goes to the right side of the same parent, 21, because 11 now fills 21's left child. Rule I3 checks the left side first and finds it taken, so rule I4 puts the new node in the empty right side.

(d) Now insert 14. Which nodes are skipped by rule I5 before a free position is found?

#	Node checked	Left child	Right child	Decision	Rule
1	5	12	8	Both children present, add to queue, move on.	I2, I5
2	12	21	3	Both children present, move on.	I5
3	8	17	9	Both children present, move on.	I5
4	21	11	2	Both children present, move on.	I5
5	3	NULL	NULL	Left is empty, so insert 14 here.	I3

Nodes skipped by I5	5, 12, 8, 21
----------------------------	--------------

(e) Use the shortcut I6 to predict, without scanning, where 6 would go next. Then check it with the full scan.

Current n	New index $= n$	Parent index $(n - 1)/2$	Parent value	Side
10	10	$(10 - 1)/2 = 4.5$, round down to 4	3	Right (even)

Full scan check:

#	Node checked	Left child	Right child	Decision	Rule
1	5	12	8	Both children present, add to queue, move on.	I2, I5
2	12	21	3	Both children present, move on.	I5
3	8	17	9	Both children present, move on.	I5
4	21	11	2	Both children present, move on.	I5
5	3	14	NULL	Right is empty, so insert 6 here.	I4

Prediction matches scan?	Yes
---------------------------------	-----

Problem 4: Deleting Elements, Every Case

Each part starts again from the original tree of Problem 1. For each deletion, find the target and the deepest node (D3) and name every rule you apply.

(a) Case A: delete 4. What happens to the tree, and why? (D3, D4)

- Target: scan level order 5, 12, 8, 21, 3, 17, 9. 4 is never found (D3, D4).
- Deepest node: 9, but it is not used.
- Result: since the target is not in the tree, nothing is deleted and the tree is unchanged (D4).
- Level order after: 5, 12, 8, 21, 3, 17, 9

(b) Case B: delete 9. Why is this the easiest case? (D7)

Drawings: Appendix, Figure 3

- Target: 9. Deepest node: 9 (D3).
- The target is the deepest node, so D5 changes nothing and D6 simply removes it: 8's right link becomes NULL (D7).
- Easiest case because no value has to be copied and no other node moves; 8 is left with only its left child, 17.
- Level order after: 5, 12, 8, 21, 3, 17

(c) Case C: delete 21, a leaf that is not the deepest node. (D5, D6)

Drawings: Appendix, Figure 3

- Target: 21. Deepest node: 9 (D3).
- Copy 9 into 21's position, the left child of 12 (D5).
- Remove the old deepest node, 9, by setting 8's right link to NULL (D6).
- Level order after: 5, 12, 8, 9, 3, 17

(d) Case D: delete 12, an internal node. Which children does the replacement node have afterwards?

Drawings: Appendix, Figure 3

- Target: 12. Deepest node: 9 (D3).
- Copy 9 into 12's position (D5), then remove the old 9 from the right of 8 (D6).
- The replacement node 9 keeps 12's children: 21 on the left and 3 on the right.

- Level order after: 5, 9, 8, 21, 3, 17

(e) **Case E: delete 5, the root. What is the new root?**

Drawings: Appendix, Figure 3

- Target: 5 (the root). Deepest node: 9 (D3).
- Copy 9 into the root position (D5), then remove the old 9 from the right of 8 (D6).
- New root: 9, with children 12 (left) and 8 (right).
- Level order after: 9, 12, 8, 21, 3, 17

(f) **Case F: the tree contains only {9}. Delete 9, then (on a fresh {9}) delete 4. (D2)**

- Delete 9 from {9}: the only node is the target, so removing it leaves an empty tree (D2).
- Delete 4 from {9}: the only node is not 4, so nothing is deleted and the tree stays {9} (D2).
- Level order after: empty (delete 9) and 9 (delete 4)

(g) **Case G: the tree is empty. Delete 5. (D1)**

- The tree has no nodes, so there is nothing to search or delete (D1).
- The tree stays empty.

(h) **Why do we move the deepest node into the target's place instead of simply removing the target?**

If we removed the target outright, we would leave a hole in the middle of the tree. Removing an internal node would also cut its children off from the rest of the tree. Either way, the tree would stop being complete (C6), and the index shortcut (children at $2i + 1$ and $2i + 2$) would no longer work. The deepest node is the last one in level order, so removing it is the only removal that leaves no gap. Copying its value into the target's place and then removing it keeps the tree complete, with every index from 0 to $n - 2$ still filled.

Summary table

Case	Delete	Target	Deepest	Rules applied	Level order after
A	4	Not found	9	D3, D4	5, 12, 8, 21, 3, 17, 9 (unchanged)
B	9	9	9	D3, D7	5, 12, 8, 21, 3, 17
C	21	21	9	D3, D5, D6	5, 12, 8, 9, 3, 17
D	12	12	9	D3, D5, D6	5, 9, 8, 21, 3, 17
E	5	5 (root)	9	D3, D5, D6	9, 12, 8, 21, 3, 17
F	9 / 4	9 / not found	9 / 9	D2	empty / 9 (unchanged)
G	5	None (empty tree)	None	D1	empty

Problem 5: Deleting All Elements

Start from the original tree and delete in the order 5, 12, 8, 21, 3, 17, 9 until the tree is empty. Do not reset the tree between deletions.

(a) For each deletion give the tree before (level order), the deepest node, the case (A to G), the rules applied and the tree after.

#	Delete	Tree before (level order)	Deepest	Case	Rules	Tree after (level order)
1	5	5, 12, 8, 21, 3, 17, 9	9	E	D3, D5, D6	9, 12, 8, 21, 3, 17
2	12	9, 12, 8, 21, 3, 17	17	D	D3, D5, D6	9, 17, 8, 21, 3
3	8	9, 17, 8, 21, 3	3	C	D3, D5, D6	9, 17, 3, 21
4	21	9, 17, 3, 21	21	B	D3, D7	9, 17, 3
5	3	9, 17, 3	3	B	D3, D7	9, 17
6	17	9, 17	17	B	D3, D7	9
7	9	9	9	F	D2	[] (empty)

(b) Draw the tree after every deletion.

Drawings: Appendix, Figure 4

(c) Which cases appeared during the sequence? Was the tree still complete after every step? Explain why.

Cases that appeared	E, D, C, B, F
---------------------	---------------

Still complete after every step? Why:

The tree stayed complete after every step. Each deletion only ever removed the deepest node, which is the last one in level order. Any other target was replaced by copying the deepest value into its place (D5). Removing the last node never leaves a gap, so the remaining nodes always filled indices 0 to $n - 1$ with no holes (C6).

Bonus: Reverse-Order Deletion

Repeat Problem 5, but delete in the reverse order: 9, 17, 3, 21, 8, 12, 5. Do not reset the tree between deletions.

Drawings: Appendix, Figure 5

#	Delete	Tree before (level order)	Deepest	Case	Rules	Tree after (level order)
1	9	5, 12, 8, 21, 3, 17, 9	9	B	D3, D7	5, 12, 8, 21, 3, 17
2	17	5, 12, 8, 21, 3, 17	17	B	D3, D7	5, 12, 8, 21, 3
3	3	5, 12, 8, 21, 3	3	B	D3, D7	5, 12, 8, 21
4	21	5, 12, 8, 21	21	B	D3, D7	5, 12, 8
5	8	5, 12, 8	8	B	D3, D7	5, 12
6	12	5, 12	12	B	D3, D7	5
7	5	5	5	F	D2	[] (empty)

(i) Which case applies at every step, and why is this order the simplest?

Case B, until the very end, which is Case F. This order is the simplest because it deletes the nodes from the highest index down to index 0. So, the target is always the last node in level order, which is the deepest node. The target never needs to be replaced. We just set its parent's link to NULL. No values are copied, nothing moves, and the tree stays complete trivially, because we only ever remove from the end.

(ii) How many values had to be copied (D5) in each order?

Deletion order	Steps that used D5	Total copies
Problem 5 (5, 12, 8, 21, 3, 17, 9)	Steps 1, 2, 3	3
Bonus (9, 17, 3, 21, 8, 12, 5)	None	0

I Used AI in the Assignment to: (Claude)

- Generate a fillable worksheet based on the provided PDF for me to fill out.
- Convert the generated Word Doc into LaTeX form for submission.
- Verify my work and answer any questions I had on the material so I could understand it better.

Appendix: Photos of Work

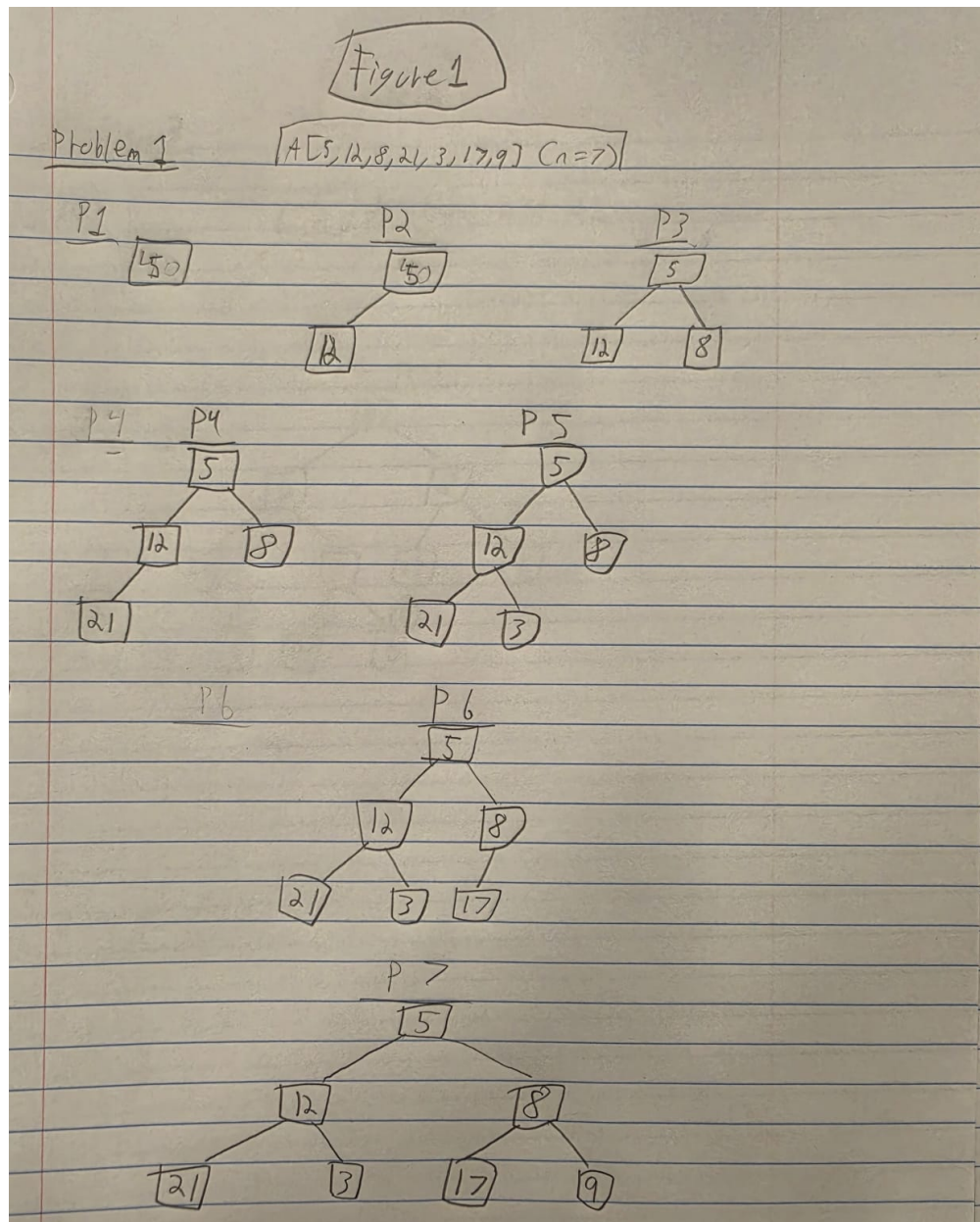


Figure 1: Problem 1 (Creating the Tree) drawings, P1 to P7: the tree after each element is added. P7 is the final tree.

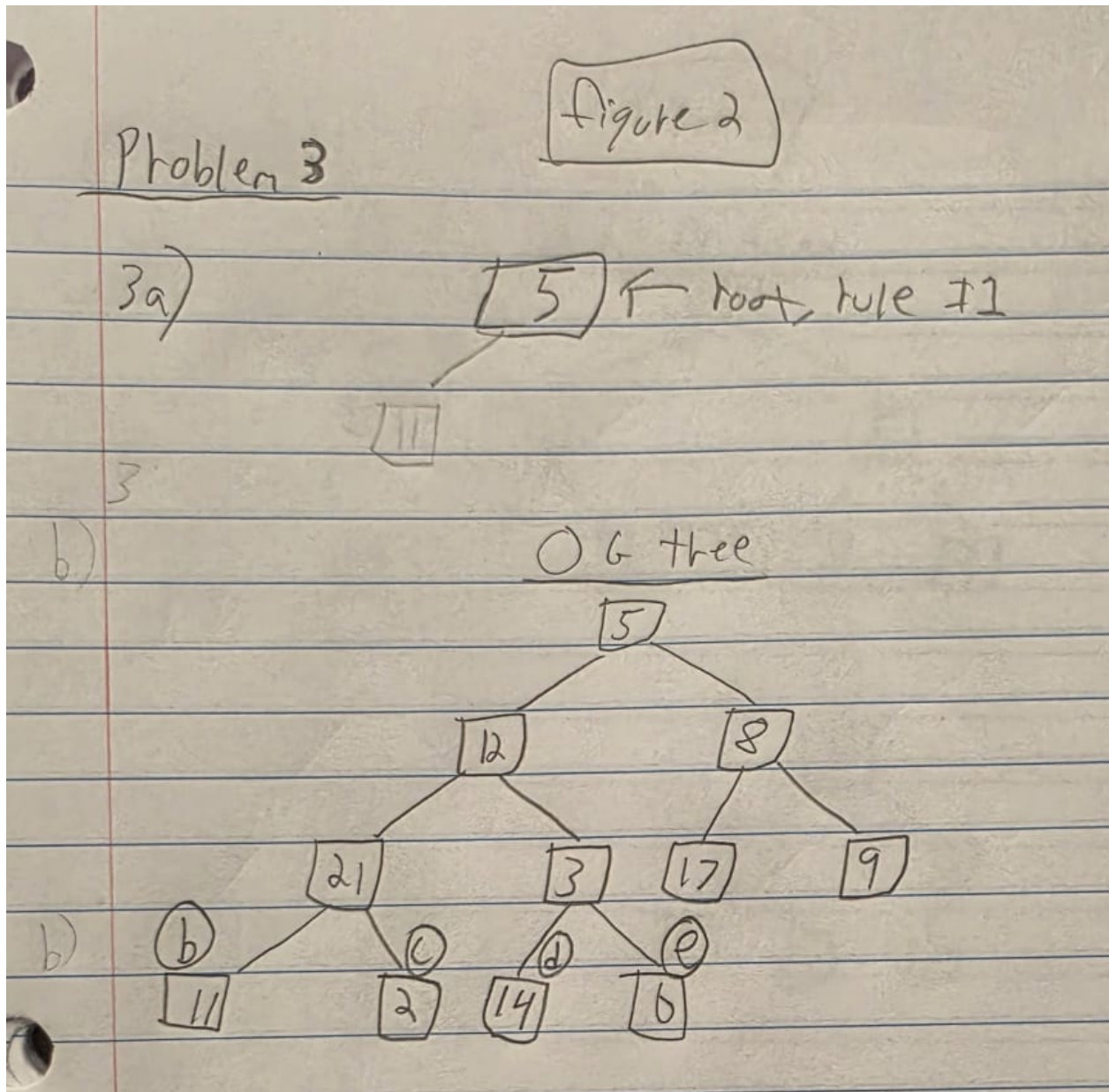


Figure 2: Problem 3 (Adding Elements to the Tree) drawings: part (a) shows 5 inserted as the root. The lower tree is the original tree with the nodes inserted in parts (b) to (e) labeled b (11), c (2), d (14), and e (6).

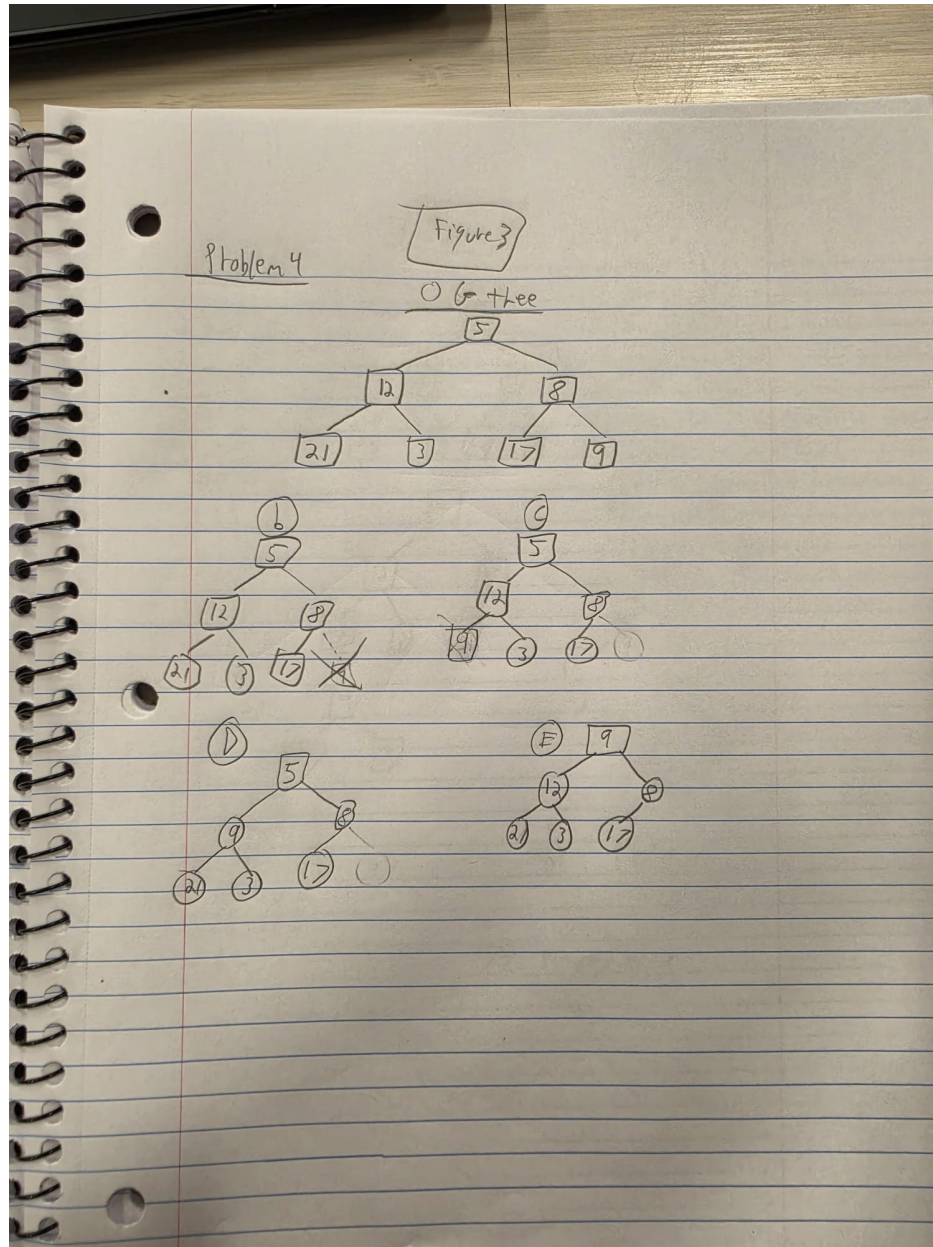


Figure 3: Problem 4 (Deleting Elements, Every Case) drawings, Cases B to E: the tree after each deletion case.

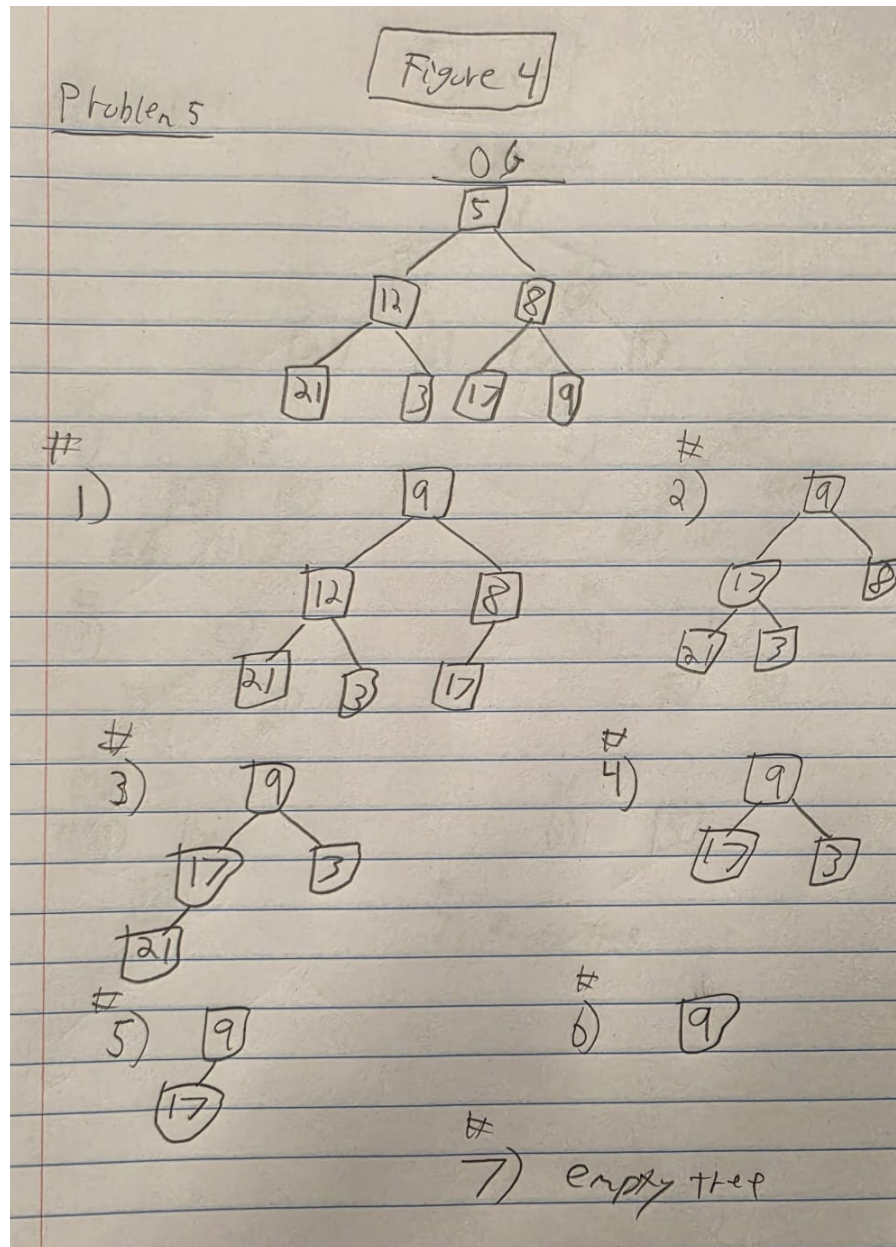


Figure 4: Problem 5 (Deleting All Elements) drawings: the original tree, then the tree after each deletion, steps 1 to 7.

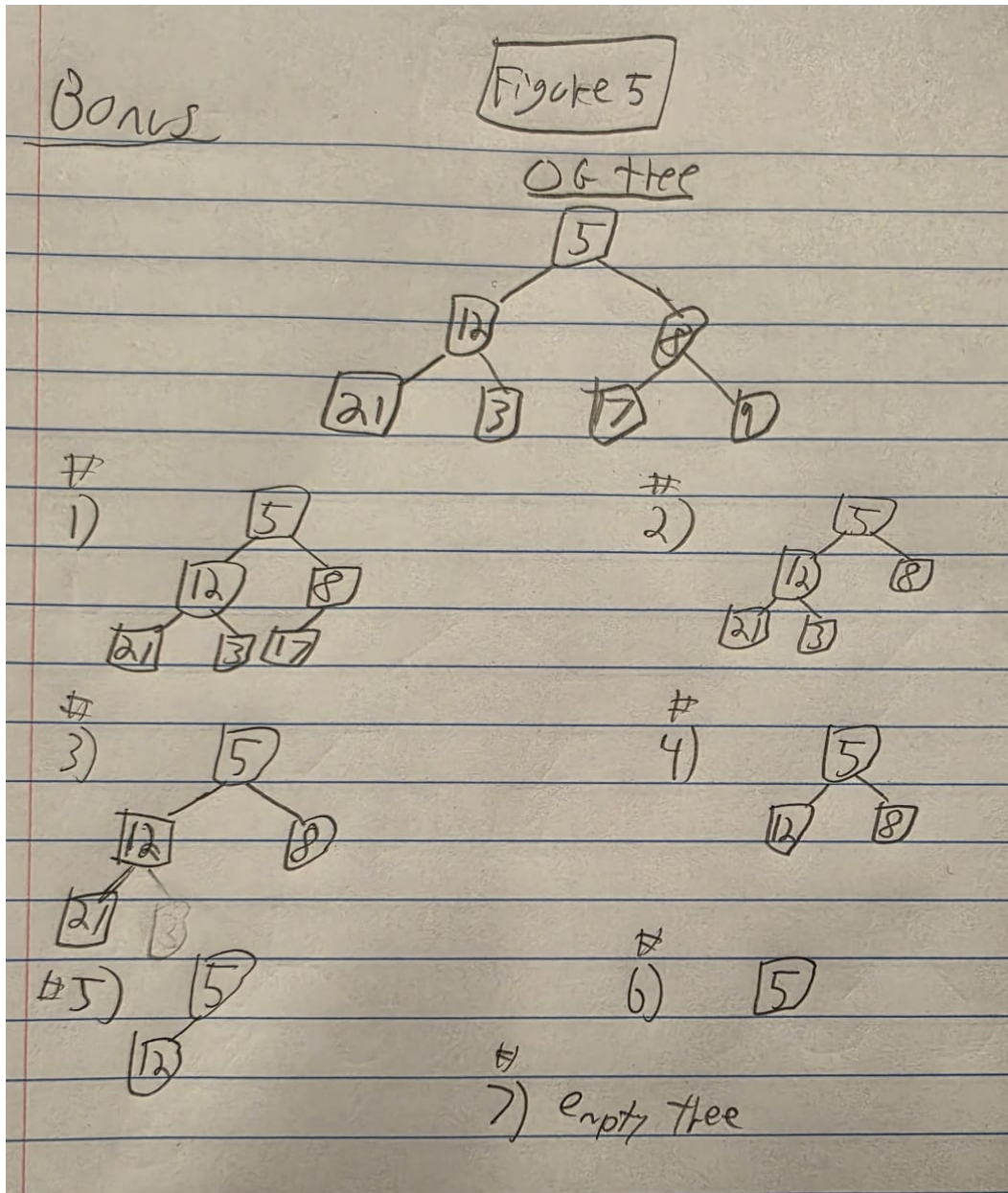


Figure 5: Bonus (Reverse-Order Deletion) drawings: the original tree, then the tree after each reverse-order deletion, steps 1 to 7.